



SC2002 OBJECT ORIENTED DESIGN & PROGRAMMING  
AY 25/26 Semester 1 Group Assignment

GitHub Main Page:

<https://github.com/act133/SC2002-IPMS-AY2526S1-Project>

| NAME               | MATRICULATION NUMBER |
|--------------------|----------------------|
| Aaron Wee Zhi Rong | U2440116B            |
| Pan Yifan          | U2440765E            |
| Loh Gan Sui        | U2440592K            |
| Daniel Chua Han    | U2440577B            |
| Yeo Swe Hon        | U2440492B            |

## Table of Contents

|                                |          |
|--------------------------------|----------|
| <b>1.Design Consideration</b>  | <b>3</b> |
| 1.1 Approach                   | 3        |
| 1.2 Assumption Made            | 3        |
| 1.3 SOLID design principle     | 3        |
| 1.4 Design Patterns            |          |
| 1.5 New features               | 3        |
| <b>2. UML Class Diagram</b>    | <b>4</b> |
| 2.1 Explanation                | 4        |
| <b>3. UML Sequence Diagram</b> | <b>5</b> |
| 3.1 Explanation                | 6        |
| <b>4. Reflection</b>           | <b>7</b> |

# 1.Design Consideration

## 1.1 Approach

The Internship Placement Management System (IPMS) is developed using a layered architectural approach to ensure high cohesion, loose coupling, and long-term maintainability. The system is organised into four logical layers: *Entities*, *Repositories*, *Controllers*, and *Boundaries*.

Entities represent the core domain objects, such as *students*, *company representatives*, *staff*, *internship postings* and *applications*, and are designed to encapsulate only data and essential state-related behaviour.

Repositories act as the storage layer, maintaining in-memory collections of these entities while ensuring persistent storage through serialized .ser files.

Controllers, which include modules such as the *LoginController*, *ApplicationService*, *PostingService*, and *ApprovalService*, implement the core business logic of the system.

They mediate all interactions between the UI boundaries and the repositories, enforcing rules such as application constraints, approval workflows, and posting visibility. Finally, the boundary layer consists of console-based user interfaces that interact with the user, delegating all requests to the appropriate controllers while avoiding direct access to system internals.

## 1.2 Assumptions Made

- 1) We assumed that the department of the staff does not matter, any staff can approve the internship opportunities and withdrawal requests.
- 2) A user can only be logged in from one session at a time.
- 3) Only 1 preferred major per internship.
- 4) Filter settings persist only within the current session, resets after the user log outs.
- 5) We assume that the maximum number of internship applications per student is limited to 3 in total, regardless of withdrawals.

## 1.3 SOLID design principle

### 1.3.1 Single Responsibility Principle (SRP)

SRP states that a class should have one and only one reason to change. In our design, this is achieved by clearly separating UI concerns, domain state, and application logic. For example, in our domain entity classes, which only store state only and are exposed to minimal behaviour related strictly to that state. *InternshipPosting* class contains basic attributes of the internship position, getters and setters, and simple internal invariants like *hasCapacity()* and *confirmOne()*. Other tasks like approve itself, validate date ranges, determine student eligibility etc are not included. All such logic belongs to service classes like *PostingServiceImpl*, *ApplicationServiceImpl*, which means changes in the internship rules or business policies do not require modifying the entity. This design maintains clarity and reduces the risk of unintended coupling.

### 1.3.2 Open Closed Principle (OCP)

The Open-Closed Principle states that software entities should be open for extension but closed for modification. In our system, this is shown in the design of the service layer. We have defined the *AuthService* interface which defines authentication behaviours like login and password changing for the users. As this is an interface, its method signatures form a stable contract that is closed for modification. Boundary classes like *StudentLoginOptions* and *StaffLoginOptions* call only these interfaces and are therefore insulated from the internal details of authentication or password verification. If any further changes are required, such as add login attempt changes, we can implement by changing or decorating only the concrete *AuthServiceImpl* without changing the service interfaces or any boundary code. As a result, the services layer remains closed for modification at its public API and call sites, yet open for extension through new implementations, satisfying the OCP.

### 1.3.3 Liskov Substitution Principle (LSP)

The Liskov Substitution Principle states that the objects of a superclass should be replaceable with objects of any of its subclasses without breaking the correctness of the system. In our design, this principle is demonstrated through the *User* inheritance hierarchy. The abstract *User* class defines the common identity attributes shared by all users, and each subclass like *Student*, *Staff* and *CompanyRep*, inherits these properties and does not violate or weaken the expected behaviour defined in the base class. For example, password changing logic or repository lookups can safely work with any specific user subtype without needing special-case handling. Therefore, subclasses and implementations respect the expectations of their base types, ensuring the substitution does not break functionality and thereby satisfying the LSP.

### 1.3.4 Interface Segregation Principle (ISP)

The Interface Segregation Principle states that clients should not be forced to depend on methods they do not use. Our system applies this principle by dividing functionality into multiple small,

focused and interfaces rather than large ones. For example, the service layer is split into specific interfaces such as *AuthService*, *PostingService*, *ApplicationService*, and *ApprovalService*, each representing a single category of operations. Boundary classes depend only on the interfaces they require, ensuring that the student login page, staff approval page, and company representative workflows are never coupled to unnecessary methods. Similarly, the repository layer uses segregated interfaces, allowing each service to depend only on the data operations it needs, therefore satisfying ISP.

### 1.3.5 Dependency Injection Principle (DIP)

The Dependency Inversion Principle states that high-level modules should depend on abstractions rather than concrete implementations. In our design, DIP is clearly demonstrated where boundary layers(classes such as *StudentLoginOptions*, *RepMainPage*, and *StaffLoginOptions*) do not communicate directly with specific service classes such as *AuthServiceImpl* or *PostingServiceImpl*. Instead, they interact only with the abstract service interfaces provided by *AppContext.ServiceRegistry*. The concrete implementations are created and connected centrally in *ServiceRegistryImpl*, which also injects the required repository interfaces.

As a result, the boundary layer remains independent of low-level details such as storage and authentication classes. This design allows changes such as replacing the password hasher or switching to a different data storage approach to be made without modifying any high-level code. This separation of concerns aligns with the DIP and improves flexibility of the system.

## 1.4 Design Patterns

### 1.4.1 Singleton Pattern

The Singleton Pattern ensures that a class only has one instance and provides a global point to it (GeeksForGeeks, 2025). In our design, we deliberately used a Singleton for our access point. In this case, *AppContext* is initialised once in *Startup.init()*, and then shared everywhere through *AppContext.services()*. This was a conscious choice to centralise repositories, hashers, clocks, etc, to ensure a consistent set of services on all screens (*StudentMainPage.java*, *StaffMainPage.java*, *RepMainPage.java*, *WelcomePage.java*, etc).

```
public final class Startup {  
  
    public static void init() {  
        Clock clock = Clock.system();  
        PasswordHasher hasher = new Sha256PasswordHasher();  
  
        UserRepository users = new SerializedUserRepository(hasher);  
        PostingRepository postings = new SerializedPostingRepository();  
        ApplicationRepository apps = new SerializedApplicationRepository();  
        ApprovalQueue queue = new SerializedApprovalQueue();  
  
        AppContext.setServices(new ServiceRegistryImpl(users, postings, apps, queue, hasher, clock));  
    }  
}
```

### 1.4.2 Facade Pattern

The Facade Design Pattern simplifies interactions with complex subsystems. It provides a unified, easy-to-use interface while hiding internal details, reducing complexity for users and promoting cleaner and more maintainable code (GeeksForGeeks, 2025). Our *Console.java* fits this description. This class is a small helper that centralises inputs and outputs by printing titles and messages, showing prompts, reading user input with one shared *Scanner*, etc. Consequently, the rest of the UI never touches *System.out* / *System.in*, making it easier to tweak formatting or prompts in one place, thereby making the code more readable and consistent. This is precisely the advantage of following the Facade Pattern.

```
public final class Console {
    private static final Scanner input = new Scanner(System.in);

    public static void title(String s){ System.out.println("\n" + s); }
    public static void say(String s){ System.out.println(s); }
    public static String ask(String prompt){ System.out.print(prompt + ": "); return input.nextLine().trim(); }
    public static void pause(){ System.out.println(x: "Press Enter to continue..."); input.nextLine(); }
```

### 1.4.3 Strategy Pattern

The Strategy Design Pattern allows us to define a family of algorithms or behaviours, put each in a separate class, and make them interchangeable at runtime (GeekForGeeks, 2025). For example, Services depend on small interfaces such as *Clock.java* and *PasswordHasher.java*. Concrete strategies (*Sha256PasswordHasher.java*, *Clock.java*, etc) are supplied at startup. As the strategy interface is only visible to the callers, we can switch algorithms such as implementing a new password hasher or swapping serialised repositories for a database by editing the code very slightly in *Startup.java* without touching the UI or logic of the program. This makes testing easier and also follows the Open-Closed Principle (being able to extend new implementations without modifying the code).

```
public interface PasswordHasher {
    String hash(String raw);
    boolean matches(String raw, String hashed);
}
```

```
public interface Clock {
    LocalDate today();

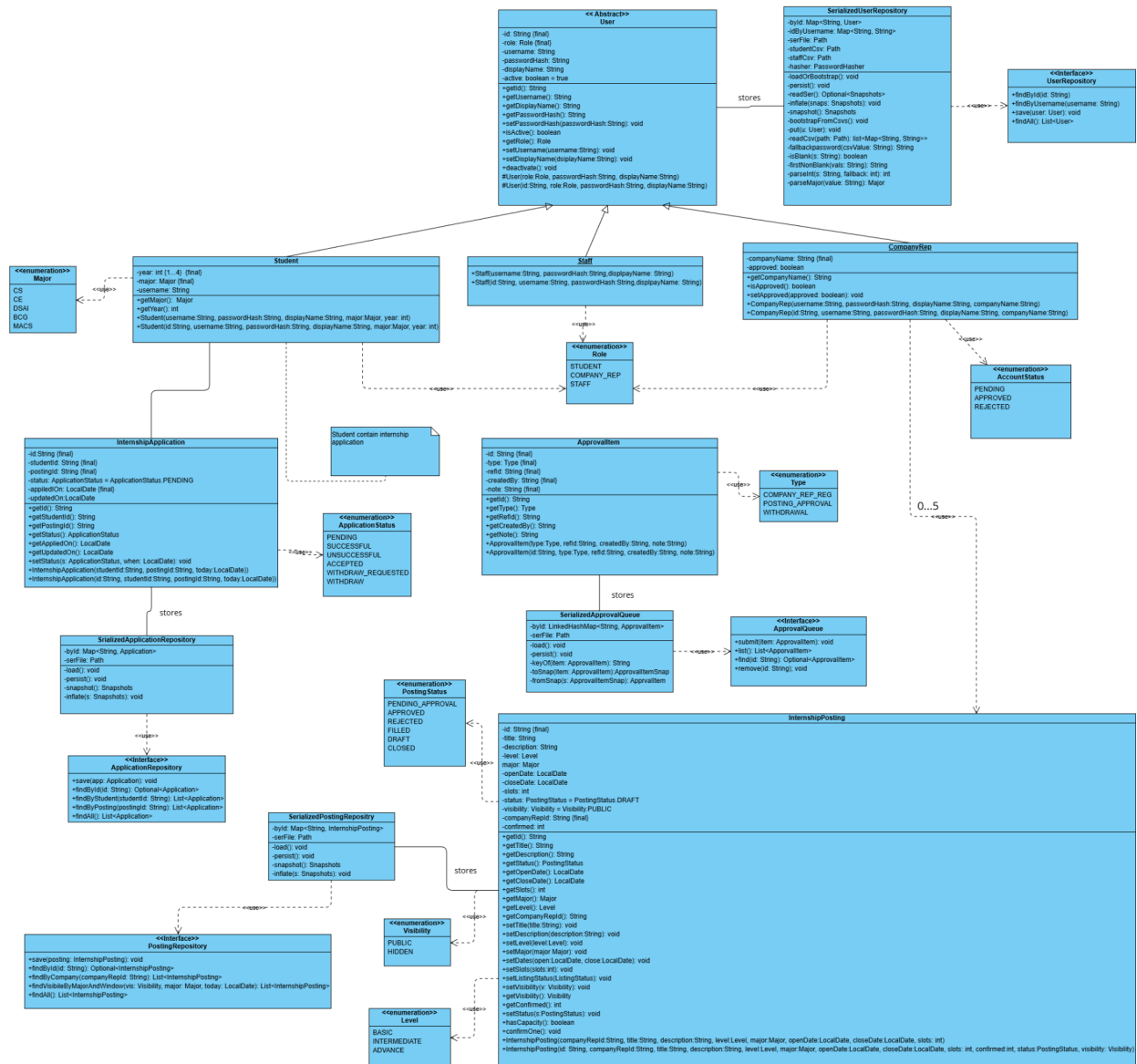
    static Clock system() {
        return LocalDate::now;
    }
}
```

## 1.5 New Features

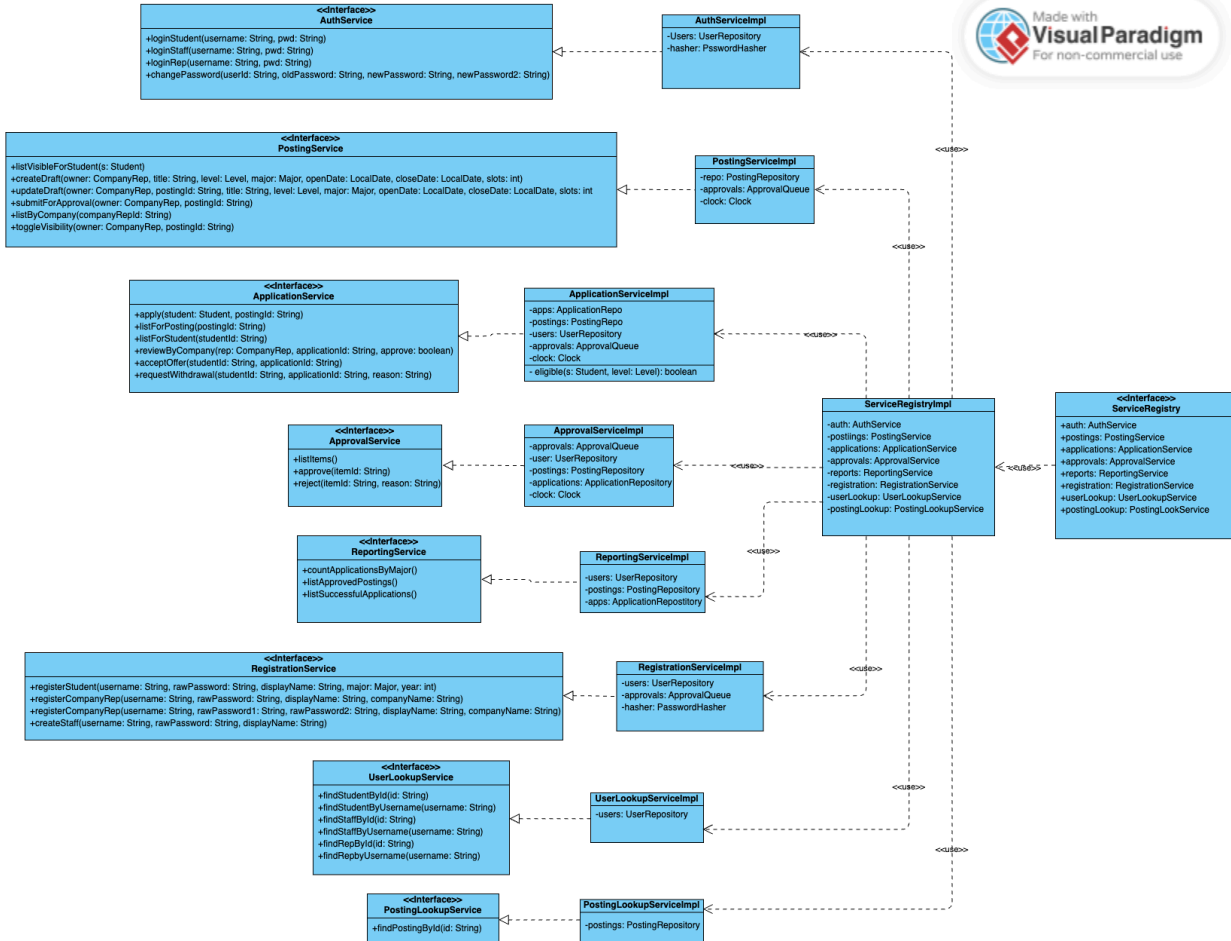
The biggest feature of our app is the Serialised Repositories of User, Postings, ApprovalItem and Application entities. Our serialization uses the “snapshot” concept where instead of serialising the entity classes, we extract their attributes and serialise them only, decoupling the entity classes from serialization storage.

## 2. UML Class Diagram

### 2.1 Entity

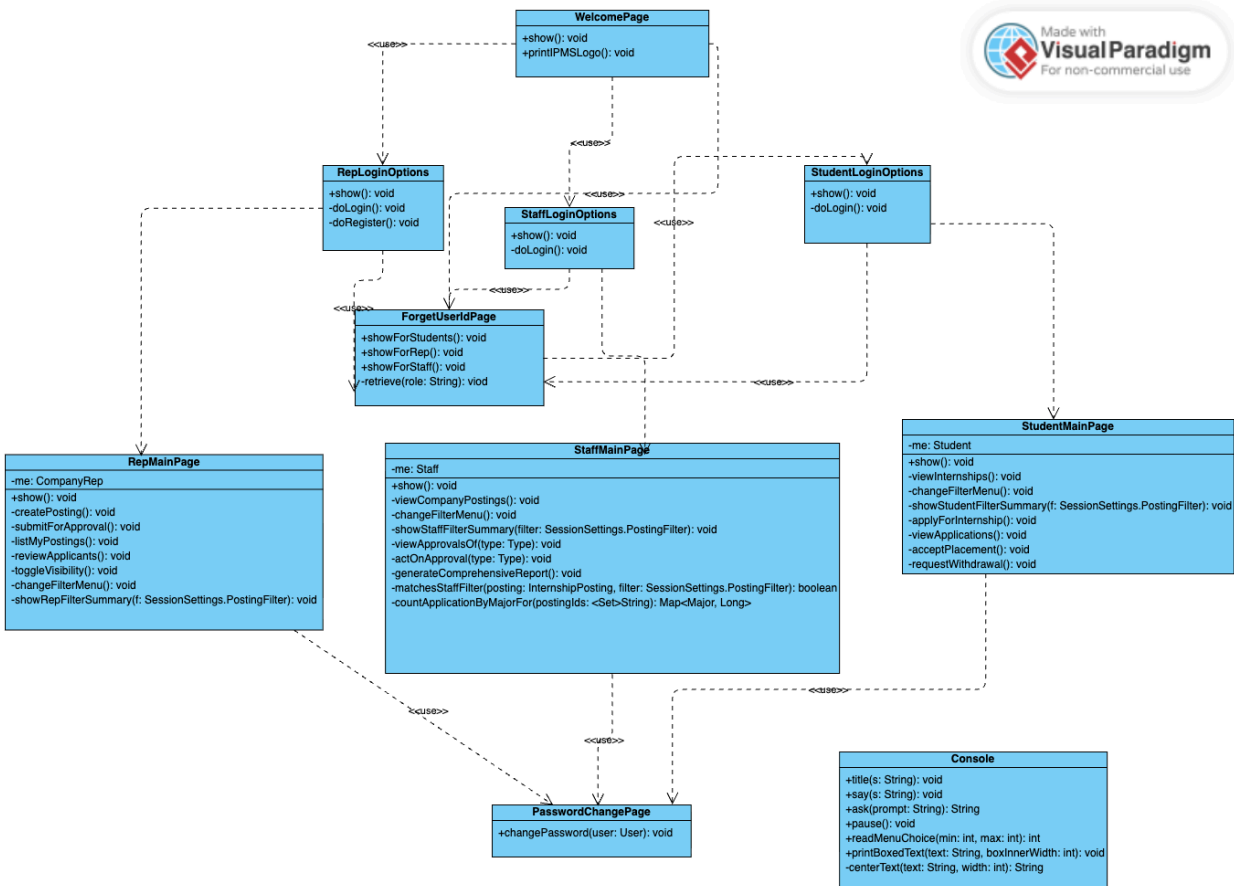


## 2.2 Control

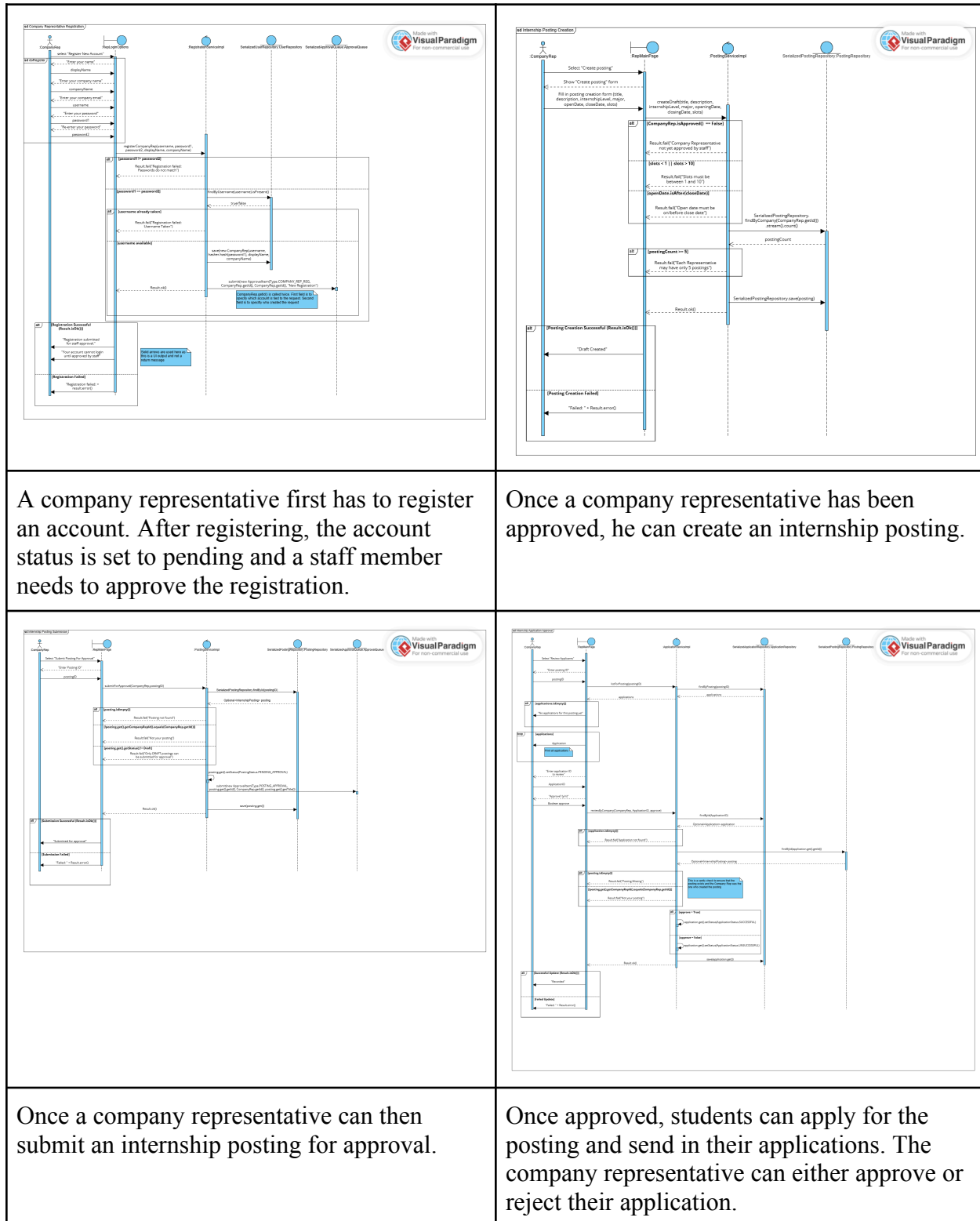




## 2.3 Boundary



### 3. UML Sequence Diagram for Company Representative



## 4. Reflection

### 4.1 Difficulties encountered and conquering it

During the development of the system, our group encountered several challenges. One major difficulty was translating the textual assignment requirements into a coherent object-oriented model. Some requirements were ambiguous, for example, the behaviour of withdrawals or visibility toggling, so we had to form consistent assumptions before proceeding. We also struggled with designing a clean separation between Boundary, Control, and Entity classes, as some functionalities initially became overly coupled. To overcome these issues, we held multiple discussions to clarify class responsibilities and iterated through several design drafts. We also relied on incremental testing to validate our logic, especially for application and approval workflows. Through team communication, constant refinement, and referencing OOP principles taught in lectures, we successfully aligned our design with the project goals.

### 4.2 Possible further enhancements

To make the system more comprehensive, we identified the potential for implementing an automatic recommendation system for students. Based on the student's major, year of study, eligibility, and the current internship posting available, the system could recommend suitable opportunities that match the student's profile. This would help students discover relevant internships more quickly and reduce the time spent manually filtering options. Similarly, Company Representatives could receive recommended applicants who fit their preferred major or internship level.

## References

<https://www.geeksforgeeks.org/system-design/singleton-design-pattern/>  
<https://www.geeksforgeeks.org/system-design/facade-design-pattern-introduction/>  
<https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/>